



Common Developer Errors & Troubleshooting Guide

Navigating the complexities of development can be challenging. This guide highlights common pitfalls and provides clear solutions to help streamline your workflow and prevent frustrating bugs.

Layout & UI Logic Errors: Building a Solid Foundation

Missing Scaffold Widget

Cause: Starting a screen with Container or Column without wrapping it in a Scaffold, leading to black screens or malformed text.

Solution: Always wrap screen content in a Scaffold for basic Material Design layout (AppBar, Body, etc.).

Color Property Outside BoxDecoration

Cause: Defining a color for a Container while also using a decoration. Flutter throws an error as color must be inside the decoration.

Solution: Move the color property inside the BoxDecoration(color: ...) block.

Incorrect Parent Widget Usage

Cause: Using widgets like Expanded or Flexible outside of a Row, Column, or Flex, which are their required direct descendants.

Solution: Ensure Expanded is only used within a Column or Row.

Scrolling & Sizing Issues: Navigating Content Flow

01

ListView Inside SingleChildScrollView

Cause: Nesting scrollable widgets without defining boundaries, leading to "Vertical viewport was given unbounded height" errors.

Solution: Set `shrinkWrap: true` and `physics: NeverScrollableScrollPhysics()` on the inner `ListView`. or use `SizeBox()`

02

Spacer/Expanded in Scrollable Widget

Cause: Placing a `Spacer` inside a `ListView` or `SingleChildScrollView`, which have infinite height/width, causing the `Spacer` to fail.

Solution: Give the parent a fixed height or use `SizeBox` with a specific height instead of `Spacer`.

03

DefaultTabController Length Mismatch

Cause: The `length` property in `DefaultTabController` doesn't match the actual number of widgets in `TabBar` or `TabBarView`.

Solution: Ensure the `length` integer strictly matches the number of tabs and views provided.

Logic & State Management: Ensuring Predictable Behavior



Unhandled States

Cause UI only handles "Success" and "Error" states in a flow (e.g., Login), neglecting "Loading" or null states, leading to crashes.

Solution Use a Switch statement or if/else in BlocBuilder to handle all possible states (Initial, Loading, Success, Failure).



SizedBox() on Login Success

Cause After a successful Firebase login, the UI returns an empty SizedBox instead of navigating to the Home Screen.

Solution Use a BlocListener to handle navigation (e.g., Navigator.push) instead of returning a widget from the builder.



Mixing Bloc and Provider

Cause Attempting to access a Bloc using Provider syntax without proper dependency injection.

Solution Use BlocProvider to provide the bloc and context.read() or BlocBuilder to access it.

Network & Firebase Setup: Connecting Your App

→ Missing Base URL or Endpoint

Cause: API requests fail because the BaseURL isn't concatenated with the specific EndPoint, or the string is empty.

Solution: Create a `Constants` class to store the BaseURL and ensure every API call properly appends the required EndPoint.

→ Files Outside the Lib Folder

Cause: Placing `.dart` files in the root project or test folder and attempting to run them as part of the main app.

Solution: Ensure all application source code is located inside the `lib/` directory.



Firebase Specific Bugs: Common Authentication & Data Issues

1

Null Check on User Properties

Cause: Trying to display `user.email` or `user.displayName` before the Firebase User is fully loaded.

Solution: Use the null-aware operator (`user?.email ?? "Guest"`) to prevent app crashes.

2

`google-services.json`
Missing/Misplaced

Cause: Adding the Firebase package but forgetting to place the configuration file in the `android/app` folder.

Solution: Download the file from the Firebase Console and place it in the correct platform directory.

3

"Permission Denied" (Firestore Rules)

Cause: Attempting to read or write to Firestore when security rules are set to "Locked Mode" or require unfulfilled authentication.

Solution: Update your Firestore/Storage rules to allow access to authenticated users (`request.auth != null`).

ADDITIONAL BUGS

Firestore & Async Initialization: Timing is Everything

Calling Firebase Services Before WidgetsFlutterBinding

Cause Initializing Firebase in `main()` without calling the binding first, causing the app to crash by trying to interact with the native platform prematurely.

Solution Always add `WidgetsFlutterBinding.ensureInitialized();` before `await Firebase.initializeApp();` in your `main()` function.

Not Awaiting Future Methods

Cause Calling Firebase methods like `FirebaseAuth.instance.signIn...` or `Firestore.collection...add()` without `await`, leading to "null" data errors as code proceeds before server response.

Solution Mark functions as `async` and use `await` for every Firebase network call.

ADDITIONAL BUGS

Logical & Memory Management: Optimizing Performance

- **"SetState called after dispose"**

Cause: Triggering `setState()` inside a `Future.delayed` or API callback after the user has navigated away from the screen.

Solution: Before calling `setState()`, check if the widget is still "mounted" using `if (mounted) { setState(...); }`.

- **Memory Leaks from Controllers**

Cause: Forgetting to close `TextEditingController`, `AnimationControllers`, or `StreamControllers` in the `dispose()` method.

Solution: Always call `controller.dispose()` when the widget is removed from the tree to save device memory.

Data Handling & Models: Ensuring Data Integrity

1

Parsing "Double" as "Int" from Firebase

Cause: In Firestore, numbers are sometimes stored as `double` (e.g., 22.0), but the Flutter model expects an `int`, causing a "Type Subtype" mismatch error.

Solution: Use `.toDouble()` or `num` type in your models to handle numbers safely from Firestore documents.

2

Missing "Key" in List Items

Cause: Reordering or deleting items in a list without providing a `ValueKey` or `UniqueKey` to list items, leading to incorrect UI updates or wrong data display.

Solution: Use a `Key` (like the document UID from Firebase) for each item in your `ListView`.

Conclusion: Mastering Development Challenges

By understanding and proactively addressing these common developer errors, you can significantly improve your development process, reduce debugging time, and build more robust and reliable applications.



Happy coding!